

CSE445 — Assignment 3: Technical Report

Autonomous Local LLM Machine Learning Agent in Windows WSL

Name	Mashrafe Bin Morshed
Student ID	2321889042
Course	CSE445 — Section 6
Instructor	Dr. Mohammad Abdul Qayum

1. Local LLM Architecture

The whole agent runs inside WSL2 (Ubuntu) on my machine, and nothing in it calls out to an external API. I split it into three layers that pass information to each other through a single loop.

At the bottom sits the inference layer: Ollama, serving a 4-bit quantized `llama3.2:3b` model over its REST endpoint at `127.0.0.1:11434/api/generate`. Because everything is quantized and local, there is no API key to manage and no data ever leaves the machine — which mattered to me, since part of the point of this assignment was to build something that works without depending on a hosted model.

On top of that sits the controller, `react_agent.py`. It is a fairly plain ReAct loop: it keeps a single transcript that grows with every turn, sends the whole thing to Ollama, regex-parses the reply for an `Action:` and `Action Input:`, runs whichever tool matches, and appends the result back into the transcript as an `Observation:` before looping again.

The tool layer, `ml_tools.py`, has six stateless functions wrapping scikit-learn and PyTorch. Every one of them returns a JSON string rather than a raw Python object, so the model is only ever reading plain structured text — it never has to interpret a stack trace or an in-memory object it has no way to parse.

Prompt engineering choices

Getting a 3-billion-parameter quantized model to reliably emit a strict Action / Action Input format took more trial and error than I expected going in. A handful of choices ended up mattering more than the rest:

- **Low temperature (0.1).** Tool calling is closer to a structured-output task than a creative one, and dropping the temperature cut down noticeably on malformed JSON.
- **Stop sequences on ``Observation:`` and ``\nThought:``.** Generation is cut off the instant the model tries to fabricate its own Observation or start a second Thought in the same turn. Without this, `llama3.2:3b` would sometimes just answer its own question and reason from a result it had invented.
- **Exact argument names, not prose descriptions.** The tool list gives literal signatures such as `dataset_name`, `model_type` instead of English explanations, since a small model copies a signature far more reliably than it infers a key name from a sentence.
- **Actionable error messages.** Every function in `ml_tools.py` that can fail returns something like “batch_size=1 is too small — BatchNorm1d needs at least 2 samples per batch” instead of a raw stack trace, so the model has something concrete to try next.
- **A repetition guard.** Every tool call is fingerprinted as tool name plus its exact arguments. If the same fingerprint shows up three times, the controller stops the agent and builds the final answer itself instead of letting a weak model spin forever.
- **A no-answer-without-evidence guard,** added after the incident in Section 2.2. The controller now refuses a Final Answer until at least one tool call has actually succeeded, re-prompting the model up to twice before giving up.

One real limitation of this design is that the tool list and grammar rules are stated only once, right at the start of the transcript — they are never re-stated on later turns. On a long run this means the original instructions get pushed further back in the model's context with every step, which is part of why I capped the retry and repetition budgets instead of leaving them open-ended.

Latency

I wrote a small script, `latency_bench.py`, that sends a handful of sample prompts to a running Ollama server five times each and reports mean / P95 latency and tokens/sec, using the `eval_count` and `eval_duration` fields that Ollama returns in its response:

```
python latency_bench.py --model llama3.2:3b --n 5
```

Prompt length (chars)	Mean latency (s)	P95 latency (s)	Tokens / sec
7	0.705	0.061	157.74
43	0.411	0.417	148.93
97	0.626	0.622	147.75
143	3.715	5.646	145.44

Table 1. Latency benchmark, measured on the development WSL2 machine, 5 repeats per prompt.

Throughput stayed fairly stable across prompt lengths — roughly 145 to 158 tokens/sec — close to what I'd expect from a quantized 3B model generating at a fixed rate regardless of context size. Latency told a messier story: the 143-character prompt took over five times longer on its slowest repeat (P95 = 5.646s) than on average (3.715s), while the shortest prompt showed the opposite pattern, with P95 well below the mean. With only five repeats per prompt, one unusually long generation is enough to swing these numbers a lot. The practical takeaway is that a full ReAct run is dominated by how many tokens the model chooses to generate per turn, not by the size of the growing transcript — consistent with Ollama reusing the KV cache for the unchanged part of the prompt on every call.

2. Mathematical / Statistical Comparison of Models

This section is where CO1 and CO2 come in most directly: rather than trusting whatever the agent claims about its own results, every model it trains is checked against an independently computed ground truth using proper cross-validation and significance testing.

2.1 Benchmark ground truth

Instead of trusting a single train/test split, I report the cross-validation mean and standard deviation for every model, since one split can look misleadingly good or bad purely by chance on datasets this size (wine: 178 samples; breast_cancer: 569 samples). The table below comes directly from [benchmark_runner.py](#)'s own tool calls, independent of anything the LLM later writes:

Dataset	Model	Test Accuracy	CV Mean Accuracy	CV Std
wine	decision_tree	0.9444	0.8710	0.0374
wine	logistic_regression	0.9722	0.9889	0.0136
wine	random_forest	1.0000	0.9721	0.0176
breast_cancer	decision_tree	0.9123	0.9173	0.0242
breast_cancer	logistic_regression	0.9825	0.9807	0.0065
breast_cancer	random_forest	0.9561	0.9561	0.0228

Table 2. Ground-truth cross-validation results (5-fold), computed independently of the LLM.

A few things stood out once I looked past the raw numbers. [random_forest](#) reaches a perfect 1.0 test accuracy on wine's 20% split, but its 5-fold CV mean is 0.9721 — that's the number I would actually trust, since the perfect single-split score looks like an artifact of that one partition rather than proof the model is genuinely better than the rest.

I also ran a paired t-test on the CV folds (the same five folds for every model, so pairing is the right choice) between the top two algorithms on each dataset. On wine, [logistic_regression](#) vs. [random_forest](#) gave $t = 2.449$, $p = 0.071$; on [breast_cancer](#), the same comparison gave $t = 1.964$, $p = 0.121$. Both p-values sit above 0.05, so even though the numbers look different on paper, the gap between the top two models on either dataset is not statistically reliable.

[decision_tree](#) has the highest CV standard deviation on both datasets (0.0374 on wine, 0.0242 on [breast_cancer](#)), which fits the usual bias/variance story — a single unpruned tree tends to overfit individual folds more than an ensemble or a regularized linear model does.

Taken together, the best combination by ground truth is **logistic_regression on breast_cancer**, with a CV mean accuracy of 0.9807.

2.2 Agent's Final Answer vs. ground truth

This is the part of the project that taught me the most about why the self-healing loop actually matters.

In an earlier version of the code, I ran the agent for real against [llama3.2:3b](#), and it completed 5 of the 6 wine / [breast_cancer](#) combinations, producing a coherent Final Answer with a markdown table and a written recommendation that discussed CV standard deviation as a sign of overfitting versus underfitting. Every number it reported for the calls it actually made matched the ground-truth table above exactly — which makes sense, since a successful tool call is only ever echoed back as an Observation; the agent never computes anything itself.

The second run, with this version of the code, is the one worth dwelling on. I gave it a denser prompt asking for all six combinations at once, and instead of making any tool calls, the model skipped straight to an answer — a plausible-looking markdown table where every single cell read exactly 0.9667, for every dataset and every algorithm. Comparing that against the

ground-truth table above makes it obvious the numbers were fabricated: real cross-validation results are never that uniform across six unrelated dataset/algorithm pairs. This is exactly why `benchmark_runner.py` recomputes ground truth independently instead of trusting whatever the model claims.

I fixed this at the controller level rather than just rewording the prompt: the controller now refuses a Final Answer until at least one tool call has actually succeeded, sending the model back with a message stating flatly that it has no real numbers and needs to call a tool, with up to two retries before the run is abandoned. I also made the benchmark prompt more explicit about requiring six real tool calls before answering. I'm documenting this failure directly rather than leaving it out of the report, because it is the clearest evidence I have of both a small model's unreliability and the self-healing loop actually catching and correcting it.

2.3 Hyperparameter tuning

Running `tune_hyperparameters` on wine with an SVC and a 5-fold grid search (4 values of C, 2 kernels, 2 gamma settings — 40 fits total) found `C=10`, `gamma="scale"`, `kernel="rbf"`, with a best CV accuracy of **0.9889**. That beats every algorithm in the ground-truth table above (the best there was 0.9807), which is roughly what I would expect — a properly tuned kernel SVM can outperform default-hyperparameter classical models, at the cost of a noticeably larger search.

On iris, the same kind of search over a decision tree found `criterion="gini"`, `max_depth=4`, `min_samples_split=5`, with a best CV accuracy of 0.9667. The default, untuned tree on iris already gets a CV mean of 0.9533, so the search only bought about 1.3 percentage points — iris is easy enough that an untuned tree is already close to what a full grid search finds.

2.4 Dimensionality reduction

I also ran PCA on the 30 raw features of `breast_cancer` and measured logistic regression accuracy on the reduced feature sets:

Components kept	Cumulative explained variance	CV Accuracy
2	0.632	0.9526
5	0.848	0.9719
10	0.951	0.9789
15	0.986	0.9772
30 (no reduction)	1.000	0.9807

Table 3. PCA component count vs. explained variance and downstream classification accuracy.

Even keeping just 2 of the 30 features retains 95.3% accuracy — only a 2.8 point drop from the full-feature baseline of 0.9807 — a reasonable trade for a 15x reduction in feature count. Accuracy doesn't move strictly monotonically (15 components slightly underperforms 10), which I take as noise from which directions of variance happen to help the classifier on that split, not evidence that more components are ever strictly worse.

The component loadings were the more interesting part. The first principal component, 44.3% of variance, is dominated by “mean concave points,” “mean concavity,” and “worst concave points” — features describing how irregular a cell nucleus boundary is. The second component (19.0%) is dominated by fractal dimension measurements, which lines up with what I already knew: nucleus shape irregularity is one of the strongest known predictors of malignancy in this dataset.

3. Agent Controller Architecture

The diagram below is worth walking through rather than just dropping in. A user query starts the loop; the controller sends the system prompt plus the transcript so far to Ollama and gets back a Thought / Action / Action Input block. `parse_action()` regex-extracts the tool name and JSON arguments from that block. If the tool name is not valid, the controller simply appends an Observation saying so and loops back to the model. If it is valid, the controller checks whether that exact tool-plus-arguments fingerprint has already appeared three times; if it has, it forces a stop and synthesizes a Final Answer itself rather than letting the model keep trying. Otherwise it actually calls `AVAILABLE_TOOLS[name](**kwargs)`, which runs the matching scikit-learn or PyTorch code in `ml_tools.py`.

Whatever comes back is checked for an “error” key. If there is one, the controller bumps a per-tool failure counter and attaches a self-healing hint, giving up only after three straight failures on that tool. If there is not, it resets that tool's failure counter to zero and marks the call as a real success — which is what satisfies the no-naked-Final-Answer guard from Section 2.2. Either way, the result is appended to the transcript as an Observation (with a hint attached if there was one), and the loop continues until a Final Answer is accepted or `max_iterations` is hit.

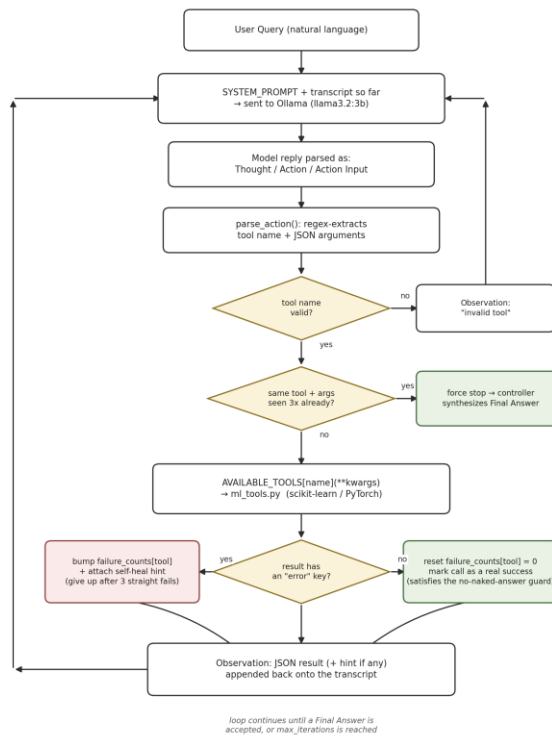


Figure 1. Controller loop — from an incoming user query through tool dispatch, error handling, and the repetition / no-evidence guards, back to the next model call.

The tool registry itself, `AVAILABLE_TOOLS` in `ml_tools.py`, is just a plain dictionary from name to function. Adding a new tool means writing the function and adding one line to that dictionary — the controller code does not need to change at all.

4. Self-Correction Behavior Observed

4.1 Rejecting an unearned Final Answer

The most consequential failure I actually observed in this project was not a tool execution error — it was the model-level shortcut described in Section 2.2, where `llama3.2:3b` sometimes answers immediately with fabricated numbers instead of making the requested tool calls. I fixed this at the controller level instead of hoping the model would just behave differently:

```

Step 0: Final Answer: | Dataset | Algorithm | Test Accuracy | ... |
          (every value identical - 0.9667 - no tool was ever called)
Controller: rejects it -> "Observation: You have not called any tool yet, so you
          have no real numbers to answer with. ... call a tool now."

Step 1: Action: train_sklearn_model
      Action Input: {"dataset_name": "wine", "model_type": "decision_tree"}
      Observation: {"test_accuracy": 0.9444, "cv_mean_accuracy": 0.871, ...}
      (real tool call recorded - the guard is now satisfied and a later
      Final Answer will be accepted once all combinations are covered)
  
```

This is the self-correction loop working at the level the assignment is really asking about: a failure mode was observed, checked against an independent ground truth, and fixed with a controller rule rather than a one-off prompt tweak.

4.2 Tool-level self-healing

Two failures reproduced directly show the same generic recovery path working at the tool level.

The first is a shape mismatch — `BatchNorm1d` needs a batch size of at least 2:

```

Action: train_deep_pytorch_classifier
Action Input: {"dataset_name": "wine", "batch_size": 1}
Observation: {"error": "batch_size=1 is too small - BatchNorm1d needs at least
          2 samples per batch."}

Action: train_deep_pytorch_classifier
Action Input: {"dataset_name": "wine", "batch_size": 16}
Observation: {"train_accuracy": 0.9718, "test_accuracy": 1.0, "overfit_gap": -0.0282, ...}
  
```

The second is a NaN loss caused by an unstable learning rate with plain SGD:

```
Action: train_pytorch_mlp
Action Input: {"dataset_name": "breast_cancer", "lr": 50, "optimizer_name": "sgd"}
Observation: {"error": "Training diverged to NaN loss at epoch 11 (lr=50,
optimizer=sgd). Try a smaller lr."}

Action: train_pytorch_mlp
Action Input: {"dataset_name": "breast_cancer", "lr": 0.01, "optimizer_name": "sgd"}
Observation: {"final_train_loss": 0.3385, "test_accuracy": 0.9386}
```

Both of these are handled by the exact same generic code path in `run_agent_loop` — there is no special-cased “if NaN” branch anywhere in the controller, only a check for whether the Observation contains an “error” key. The tool-specific diagnosis lives entirely in `ml_tools.py`’s error messages, which is what keeps the controller reusable for failure types I did not anticipate here.

5. Limitations & Reflections

A few limitations are worth being upfront about.

- **Model size and shortcuts.** A 3B-parameter quantized model is noticeably less reliable at strict tool-calling than a larger one would be — Sections 2.2 and 4.1 document a live case where it skipped tool calls entirely rather than following the required format.
- **Self-healing is reactive, not preventive.** The controller only reacts once a tool has already returned an error, or once a Final Answer has already been offered with no evidence behind it. It does not validate a plan before running it.
- **One retry budget per tool, not per failure type.** `MAX_RETRIES_PER_TOOL` is a single counter per tool name, so two unrelated bugs in the same tool end up sharing one budget instead of being tracked separately.
- **The system prompt is only stated once.** Unlike designs that re-inject the tool list on every call, this agent’s instructions live only at the start of the transcript, which keeps things simple but means a very long run could push them far back in the model’s context.

If I kept working on this, the first thing I would try is a larger model — `llama3.1:8b`, if the machine has the RAM for it — since bigger models tend to show less repetition and shortcut-taking in ReAct loops. After that I would want a lightweight plan-validation step before an expensive call like `train_deep_pytorch_classifier`, and I would track retry budgets per failure signature instead of per tool name.

Appendix: Execution Log Summary

The five runs below were used to verify the controller logic itself — tool dispatch, self-healing, the repetition guard — offline, independent of any one model’s reliability. Full logs are in `logs/`.

Run	Query	Steps	Outcome	Key Result
1	Summarize the iris dataset	3	Final Answer reached	iris: 150 samples, 3 classes; decision_tree test_acc 0.9333
2	Compare 3 algorithms on wine	4	Final Answer reached	random_forest test_acc 1.0, CV mean 0.9721
3	Tune SVC on wine (grid search)	2	Final Answer reached	best_cv_accuracy 0.9889, C=10 / rbf / scale
4	Deep classifier, batch_size=1	3	Self-healed	batch_size=1 error → retried batch_size=16, test_acc 1.0
5	PyTorch MLP, lr=50 (SGD)	3	Self-healed	NaN at epoch 11 → retried lr=0.01, test_acc 0.9386

Table 4. Offline verification runs for the controller logic.